Caracterização dos 1.000 Repositórios Mais Populares do GitHub: Um Estudo Empírico Observacional

1 Introdução

1.1 Contextualização

O desenvolvimento de sistemas de código aberto (open-source) transformou a maneira como o software é construído e mantido globalmente. O GitHub, sendo a principal plataforma de hospedagem de código, possui uma vasta quantidade de dados sobre a evolução desses projetos. Compreender como os repositórios mais populares do mundo operam e são desenvolvidos nos permite identificar padrões de sucesso e maturidade na engenharia de software.

1.2 Problema foco do experimento

O problema central consiste em identificar e analisar as principais características de sistemas populares open-source hospedados no GitHub. Busca-se entender a dinâmica de desenvolvimento desses projetos, abrangendo a maturidade temporal, a frequência de contribuições externas, o ritmo de atualizações e lançamentos, a gestão de issues e as linguagens de programação predominantes.

1.3 Questões-Pesquisa

O estudo é guiado pelas seguintes questões de pesquisa (RQs):

- **RQ 01:** Sistemas populares são maduros/antigos?
- **RQ 02:** Sistemas populares recebem muita contribuição externa?
- **RQ 03:** Sistemas populares lançam releases com frequência?
- **RQ 04:** Sistemas populares são atualizados com frequência?
- **RQ 05:** Sistemas populares são escritos nas linguagens mais populares?
- **RQ 06:** Sistemas populares possuem um alto percentual de issues fechadas?
- **RQ 07 (Bônus):** Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

1.4 Hipótese(s)

Com base na observação empírica do ecossistema open-source, foram formuladas as seguintes hipóteses:

- **H1 (Maturidade):** Espera-se que os sistemas mais populares sejam maduros, com vários anos de desenvolvimento, refletindo a necessidade de tempo para acumular visibilidade e comunidade.
- **H2 (Contribuição externa):** Espera-se que projetos populares apresentem alta taxa de contribuição externa (pull requests aceitas), indicando uma comunidade ativa de colaboradores.
- **H3 (Releases):** Espera-se que a maioria dos projetos populares possua um histórico significativo de releases formais, sinalizando práticas de entrega contínua.

- **H4 (Atualização):** Espera-se que projetos populares sejam frequentemente atualizados, refletindo manutenção ativa.
- **H5 (Issues):** Espera-se que projetos populares possuam alto percentual de issues fechadas, indicando capacidade de resposta da equipe de manutenção.
- **H6 (Linguagens):** Espera-se que as linguagens que ocupam o **topo do ranking do GitHub Octoverse 202** (GitHub, 2025) — dominem entre os repositórios mais estrelados.

1.5 Objetivo (principal e específicos)

Objetivo principal: Coletar e analisar métricas específicas dos 1.000 repositórios com maior número de estrelas no GitHub para caracterizar e compreender os padrões de desenvolvimento dos projetos open-source mais populares.

Objetivos específicos:

1. Implementar um script de mineração de dados utilizando a API GraphQL do GitHub para coleta automatizada dos dados.
2. Sumarizar os dados obtidos por meio de valores medianos, quartis e contagens por categoria.
3. Gerar visualizações gráficas para cada questão de pesquisa.
4. Confrontar os resultados com as hipóteses iniciais e com trabalhos correlatos na literatura.

---

2 Metodologia

Este trabalho caracteriza-se como um estudo empírico observacional, com análise quantitativa descritiva de repositórios de software hospedados no GitHub. A estratégia adotada concentrou-se na extração automatizada de dados públicos por meio da API GraphQL do GitHub,

seguida do armazenamento estruturado dos resultados e posterior análise estatística.

2.1 Passo a passo do experimento

O experimento foi executado nas seguintes etapas sequenciais:

Etapa 1 — Definição das questões de pesquisa: Foram estabelecidas sete questões de pesquisa (RQ01–RQ07), cada uma associada a uma métrica observável no GitHub. Essa definição orientou a escolha dos campos da query GraphQL e delimitou quais dados deveriam ser extraídos de cada repositório.

Etapa 2 — Construção da consulta GraphQL: Foi elaborada a consulta GraphQL com foco nos repositórios mais populares da plataforma, utilizando o critério `stars:>1000 sort:stars-desc`. A consulta inclui, para cada repositório, atributos como nome, URL, data de criação (`createdAt`), data da última atualização (`updatedAt`), linguagem primária, quantidade de releases, total de pull requests aceitas (estado `MERGED`) e totais de issues abertas e fechadas.

Etapa 3 — Implementação da paginação: Como a API não retorna os 1.000 repositórios em uma única resposta, foi implementado um mecanismo de paginação baseado em cursor. A cada requisição, o sistema consulta `pageInfo.endCursor` e `hasNextPage` para determinar se existe nova página disponível e, em caso afirmativo, utiliza o cursor retornado para a próxima busca.

Etapa 4 — Execução da coleta automatizada: O programa instancia o fetcher escolhido, realiza a coleta das páginas configuradas (10 repositórios por página, 100 páginas), padroniza os dados recebidos e acumula os registros em memória durante a execução.

Etapa 5 — Exportação dos dados: Ao final da coleta, os repositórios processados são exportados para arquivo CSV. Esse arquivo consolida todos os campos necessários às questões de pesquisa e serve como base para as análises estatísticas e visuais.

Etapa 6 — Análise estatística e geração de visualizações: A partir do CSV, scripts de análise calculam estatísticas descritivas (mediana, quartis, IQR, contagens) e geram gráficos para cada RQ.

2.2 Decisões

Durante a execução do experimento, algumas decisões metodológicas e técnicas foram tomadas:

Decisão	Justificativa
Tamanho da página = 10	Páginas maiores (25, 40, 50) causavam erros `502` na API. 10 repos/página × 100 páginas equilibra robustez e desempenho.
Exclusão RQ06	40 repositórios sem nenhuma issue foram excluídos do cálculo percentual, resultando em 960 repositórios válidos.
Referência temporal	Todos os cálculos de idade e dias desde atualização usam **13 de março de 2026** como data base.
Ranking de linguagens populares (RQ05 / RQ07)	Adotou-se como referência externa o **GitHub Octoverse 2025** (GitHub, 2025), cujo Top-10 de linguagens é: TypeScript, Python, JavaScript, Java, C#, PHP, Shell, C++, HCL e Go.

2.3 Materiais utilizados

- **Plataforma GitHub** como fonte dos dados observados.
- **API GraphQL do GitHub** para consulta estruturada dos repositórios.
- **GitHub CLI (`gh`)** e cliente HTTP em Python como alternativas de execução da coleta.
- **Linguagem Python** para automação da coleta, tratamento das respostas e exportação dos dados.
- Bibliotecas **`requests`**, **`python-dotenv`** e **`rich`** para comunicação HTTP, gerenciamento de variáveis de ambiente e formatação de saída em console.
- Consulta GraphQL.
- Scripts de coleta.
- Scripts de análise.
- Arquivo CSV de saída`.

2.4 Métodos utilizados

Foi utilizado o método de **Mineração de Repositórios de Software (MSR — Mining Software Repositories)** aliado a requisições de rede paginadas para a coleta de dados.

Para a análise quantitativa, adotou-se **estatística descritiva**: a sumarização dos dados numéricos contínuos e discretos utiliza a **mediana** como medida de tendência central (por ser robusta a outliers), complementada por **quartis (Q1, Q3)** e **intervalo interquartil (IQR)** para caracterização da dispersão. Os dados categóricos foram avaliados por **contagem simples e proporções percentuais**.

2.5 Métricas e suas Unidades

As métricas observadas no experimento foram definidas a partir das questões de pesquisa. O quadro a seguir sintetiza cada métrica, sua descrição operacional e unidade de medida.

Questão	Métrica	Descrição operacional	Unidade
RQ01	Idade do repositório	Diferença entre a data de coleta (`collectedAt`) e `createdAt`	anos
RQ02	Pull requests aceitas	Quantidade total de pull requests com estado `MERGED`	contagem (inteiro)
RQ03	Releases	Quantidade total de releases registradas no repositório	contagem (inteiro)
RQ04	Frequência de atualização	Diferença entre a data de coleta (`collectedAt`) e a data da última atualização (`updatedAt`)	dias
RQ05	Linguagem primária	Linguagem principal associada ao repositório pela API do GitHub	categoria nominal
RQ06	Percentual de issues fechadas	Razão entre issues fechadas e total de issues:	percentual (%)

Questão	Métrica	Descrição operacional	Unidade
		(abertas + fechadas) × 100	
RQ07	Métricas cruzadas por linguagem	RQ02, RQ03, RQ04 segmentadas por linguagem primária; comparação Top-10 Octoverse × demais	contagem, dias

Nota RQ04: `days_since_update = 0` indica atualização no dia da coleta.

Nota RQ06: Repositórios com 0 issues (40 ao total) foram excluídos, restando 960.

3 Resultados e Discussão

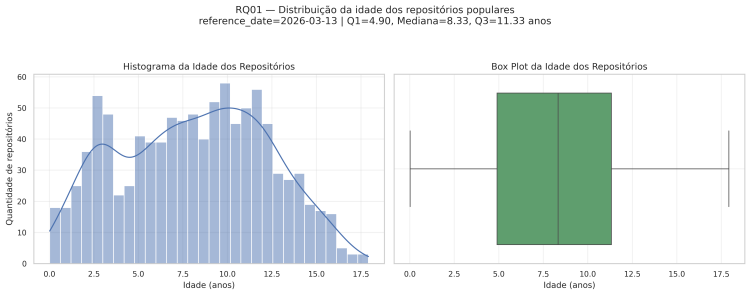
Os resultados desta seção foram consolidados a partir dos **1.000 repositórios válidos** coletados com referência temporal reproduzível em `13 de março de 2026`. Para cada questão de pesquisa, apresentam-se as estatísticas descritivas, visualizações gráficas, discussão interpretativa e, quando aplicável, análises complementares.

3.1 RQ01 — Sistemas populares são maduros/antigos?

A análise da distribuição de idade confirma a Hipótese 1 (H1), revelando que a consolidação de um repositório no

GitHub demanda tempo. Com uma mediana de 8,33 anos e o terceiro quartil atingindo 11,33 anos, fica evidente que o sucesso não é imediato. Projetos populares são, em sua esmagadora maioria, maduros, necessitando de quase uma década de desenvolvimento contínuo para construir uma base de usuários, estabelecer confiança e acumular a comunidade necessária para alcançar o topo do ranking.

Visualizações — RQ01



Distribuição de idade dos repositórios

Discussão RQ01

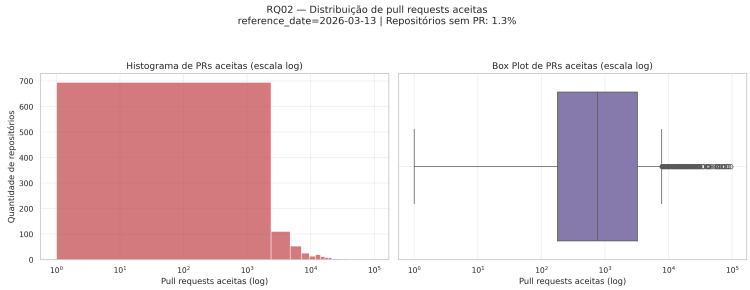
Confronto com H1: A hipótese previa que os sistemas mais populares seriam maduros, com vários anos de desenvolvimento. Os dados **confirmam H1**: a mediana de idade é de **8,33 anos**, o que indica que os repositórios do Top-1000 foram, em sua maioria, criados entre 2015 e 2018. A concentração no intervalo de 5 a 11 anos (Q1–Q3) reforça que o acúmulo de estrelas é um processo gradual — repositórios precisam de tempo considerável para construir comunidade e reputação.

3.2 RQ02 — Sistemas populares recebem muita contribuição externa?

A Hipótese 2 (H2) é fortemente validada pelos dados, visto que apenas 1,3% dos repositórios não possuem Pull Requests aceitas. A concentração de dados (mesmo em escala logarítmica) demonstra que a imensa maioria dos sistemas recebe um volume massivo de contribuições externas. Isso consolida a premissa de que o modelo

descentralizado (pull-based development) não é apenas comum, mas um pilar essencial de escalabilidade e manutenção para repositórios de alto impacto.

Visualizações — RQ02



Distribuição de pull requests aceitas

Discussão RQ02

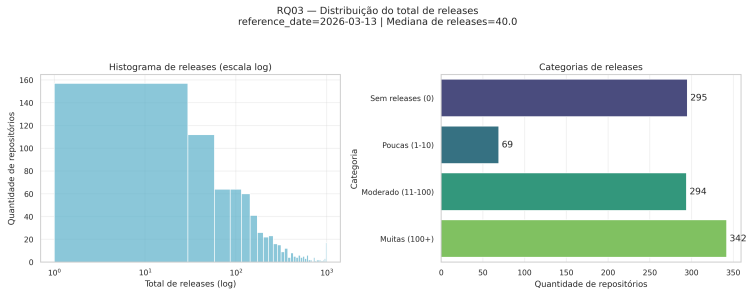
Confronto com H2: A hipótese esperava alta taxa de contribuição externa. Os dados **confirmam H2**: a mediana de **738 PRs aceitas** é um valor expressivo, indicando que a maioria dos projetos populares conta com centenas de contribuições incorporadas ao código-base. A amplitude interquartil é notável (Q1 ≈ 172, Q3 ≈ 3 201), isso demonstra que, embora a maioria dos repositórios receba um volume significativo de PRs, existe uma variação substancial, com alguns projetos acumulando milhares de contribuições, enquanto outros se mantêm em patamares mais modestos

---

3.3 RQ03 — Sistemas populares lançam releases com frequência?

A adoção de releases apresenta um comportamento polarizado que responde parcialmente à Hipótese 3 (H3). Enquanto a maior categoria é a de repositórios com “Muitas (100+)” releases (342 projetos, indicando fortes práticas de entrega contínua), um grupo expressivo de 295 repositórios não possui nenhuma release.

Visualizações — RQ03



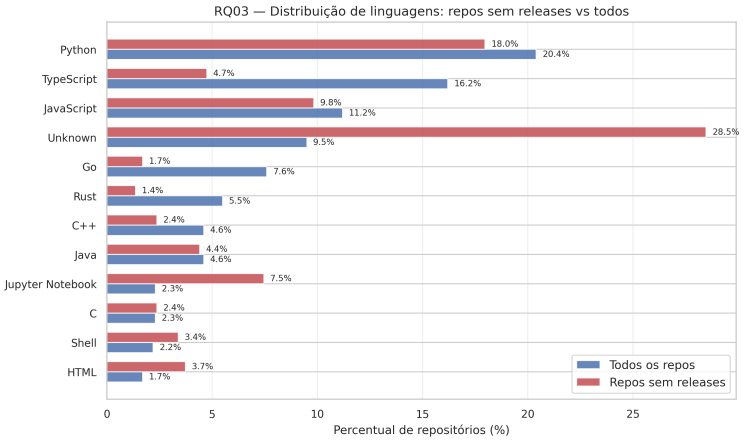
Distribuição de releases

Discussão RQ03

Confronto com H3: A hipótese esperava que a maioria possuísse histórico significativo de releases formais. Os dados **confirmam parcialmente H3**: a mediana é positiva (≈ 40), porém há uma **polarização** expressiva — quase 30 % dos repositórios não possuem nenhuma release, enquanto mais de 34 % possuem acima de 100. Isso indica que o lançamento de releases formais é uma prática que depende fortemente do tipo de projeto e do ecossistema: projetos Go e TypeScript, por exemplo, adotam versionamento semântico e publicam em registros de pacotes (npm, Go Modules), enquanto awesome lists, tutoriais e repositórios de documentação não utilizam releases como mecanismo de distribuição.

Análise complementar: Repositórios sem releases

Para compreender melhor o grupo de **295 repositórios** sem nenhuma release, foram conduzidas análises sobre sua composição por linguagem e perfil.



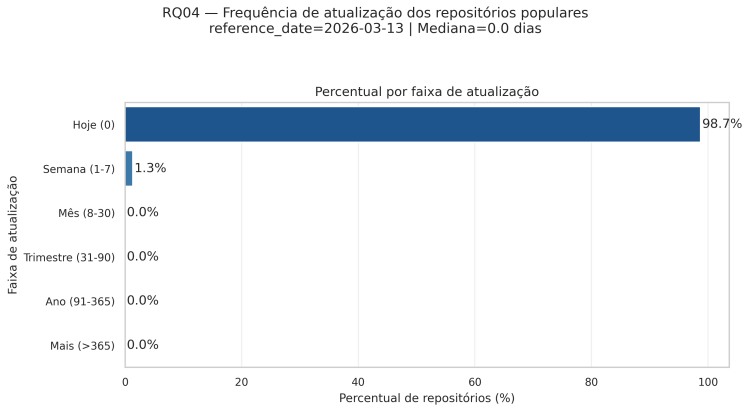
Linguagens em repositórios sem releases

Os 295 repositórios sem releases apresentam sobre-representação de categorias como *Unknown* (sem linguagem detectável) e *Jupyter Notebook*, indicando que se tratam majoritariamente de coleções de documentação, dados, awesome lists ou tutoriais — projetos que, por natureza, não utilizam releases formais. Entre os 30 mais estrelados sem releases, observam-se tanto projetos de curadoria quanto projetos de software que adotam estratégias alternativas de distribuição (Docker images, `pip`, `npm` sem releases no GitHub).

3.4 RQ04 — Sistemas populares são atualizados com frequência?

A Hipótese 4 (H4) é confirmada de forma contundente: o ecossistema de repositórios populares é extremamente dinâmico e ativamente mantido. Impressionantes 98,7% dos projetos apresentaram atualizações no mesmo dia da coleta (mediana de 0,0 dias), e todo o restante (1,3%) foi atualizado no intervalo de uma semana. Isso demonstra inequivocamente que a alta visibilidade na plataforma exige e reflete uma atividade ininterrupta de manutenção diária.

Visualizações



Frequência de atualização

Discussão

Confronto com H4: A hipótese esperava atualização frequente. Os dados **confirmam fortemente H4**: impressionantes **98,7 %** dos repositórios foram atualizados no exato dia da coleta. Os 13 repositórios restantes ocupam posições intermediárias a inferiores no ranking de estrelas (posições 365–828), sugerindo que a manutenção quase diária é um pré-requisito implícito para permanecer no topo absoluto de popularidade.

3.5 RQ05 — Sistemas populares são escritos nas linguagens mais populares?

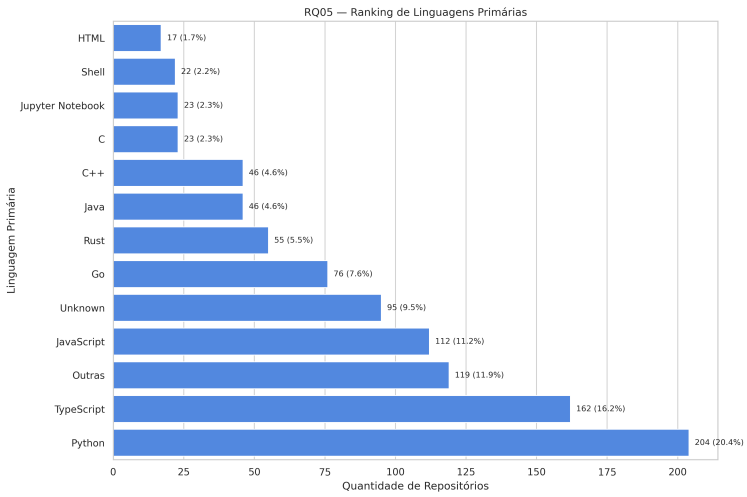
Validando a Hipótese 6 (H6), os repositórios mais estrelados são dominados pelas linguagens mais demandadas pela indústria contemporânea: Python (20,4%), TypeScript (16,2%) e JavaScript (11,2%). O protagonismo isolado do Python reflete a forte expansão de ecossistemas de Inteligência Artificial e Ciência de Dados. Paralelamente, o TypeScript assumindo a segunda posição geral indica uma forte consolidação da comunidade em adotar tipagem estática para garantir maior robustez em projetos de larga escala.

Estatísticas descritivas

Tabela 8 — Ranking das 12 linguagens primárias mais frequentes

Rank	Linguagem	Repositórios	%
1	Python	204	20,4%
2	TypeScript	162	16,2%
3	Outras	119	11,9%
4	JavaScript	112	11,2%
5	Unknown	95	9,5%
6	Go	76	7,6%
7	Rust	55	5,5%
8	C++	46	4,6%
9	Java	46	4,6%
10	Jupyter Notebook	23	2,3%
11	C	23	2,3%
12	Shell	22	2,2%
13	HTML	17	1,7%

Visualizações



Ranking de linguagens primárias

Discussão

Confronto com H6: A hipótese esperava que as linguagens do topo do **GitHub Octoverse 2025** dominassem entre os repositórios mais estrelados.

Os dados **confirmam parcialmente H6**, com ressalvas importantes:

1. **Python, TypeScript e JavaScript** ocupam as três primeiras posições tanto no Top-1000 de estrelas quanto entre as cinco primeiras do Octoverse, embora a **ordem difira**: Python lidera no Top-1000 (20,4 %), enquanto Typescript lidera no Octoverse.

2. **Go** apresenta sobre-representação expressiva: é a 5ª linguagem no Top-1000 (7,6 %), mas apenas a 10ª no Octoverse.

3. **Rust**, embora **fora do Top-10 do Octoverse**, aparece como 6ª linguagem no Top-1000 (5,5 %).

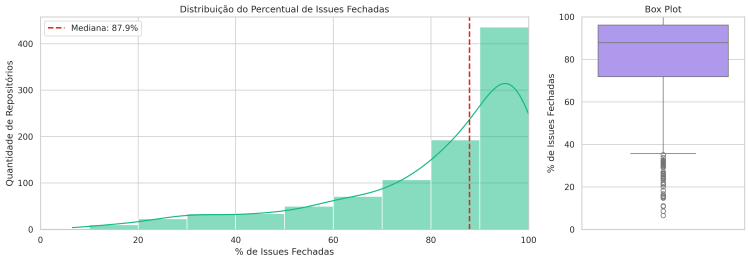
4. **C# e PHP** estão **sub-representados** no Top-1000 em relação ao Octoverse (C# é #5 no Octoverse mas não aparece no Top #10 do Top-1000; PHP é #7 mas praticamente ausente do topo de estrelas).

Conclusão As linguagens mais populares do Octoverse de fato dominam o Top-1000 de estrelas (~80 % dos repositórios são escritos em linguagens do Top-10), porém a **ordem de predominância** difere, com destaque para a sobre-representação de Python (efeito IA), Go (efeito cloud-native) e Rust (efeito hype/engajamento comunitário).

3.6 RQ06 — Sistemas populares possuem um alto percentual de issues fechadas?

A capacidade de resposta das equipes de manutenção é alta, o que confirma a Hipótese 5 (H5). A distribuição estatística é visivelmente assimétrica à esquerda, com uma mediana de 87,9% de issues fechadas e a grande maioria dos projetos concentrada na faixa de 80% a 100% de resolução. Essa alta taxa de fechamento é um forte indicador de maturidade na governança, sugerindo que projetos de sucesso possuem processos rigorosos de triagem e resolução de bugs para manter a comunidade engajada.

Visualizações - RQ06



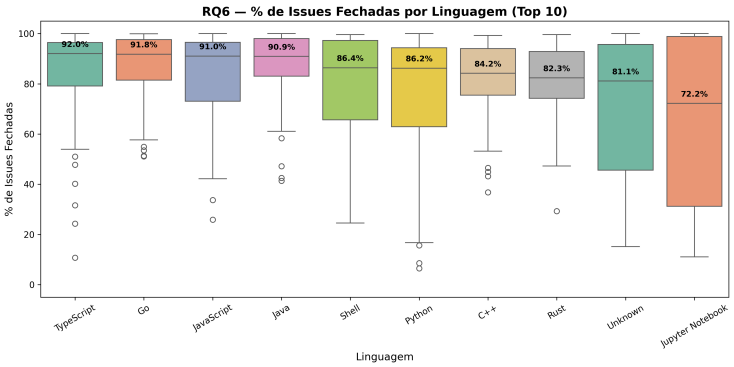
Percentual de issues fechadas

Discussão - RQ06

Confronto com H5: A hipótese esperava alto percentual de issues fechadas. Os dados **confirmam H5**: a mediana de **87,88 %** indica que o repositório popular típico resolve quase 9 em cada 10 issues reportadas. A segmentação por linguagem revela variação: TypeScript (92,0 %), Go (91,8 %), JavaScript (91,0 %) e Java (90,9 %) ficam acima da mediana geral, enquanto Jupyter Notebook (72,2 %) e Unknown (81,1 %) ficam abaixo — reflexo do perfil distinto desses projetos (documentação/curadoria, com issues menos estruturadas).

Esse resultado sugere que a **capacidade de triagem e fechamento de issues é uma característica definidora** de projetos populares, independentemente da linguagem.

Análise complementar: Relação com linguagem, idade e contribuidores



Percentual de issues fechadas por linguagem

A segmentação por linguagem revela que TypeScript (**92,0%**), Go (**91,8%**), JavaScript (**91,0%**) e Java (**90,9%**) apresentam as maiores medianas de fechamento de issues, enquanto Jupyter Notebook (**72,2%**) e Unknown (**81,1%**) ficam abaixo da mediana geral. Essa diferença pode refletir tanto a natureza dos projetos associados a cada linguagem quanto a maturidade dos processos de triagem adotados por suas comunidades.

3.7 RQ07 (Bônus) — Sistemas escritos em linguagens mais populares recebem mais contribuição externa, lançam mais releases e são atualizados com mais frequência?

A síntese transversal dos dados evidencia que a linguagem primária atua como um forte indicativo do comportamento e da dinâmica de engenharia da comunidade (RQ07). Observou-se que ecossistemas mais modernos, como TypeScript e Rust, destacam-se pelo altíssimo volume de

colaboração externa (maiores medianas de Pull Requests), enquanto linguagens como TypeScript e Go lideram amplamente a adoção de entregas frequentes (maiores medianas de releases). Tais constatações comprovam que as ferramentas e a cultura idiomática de cada linguagem influenciam diretamente as métricas de evolução e manutenção do repositório.

---

4 Insights Adicionais

4.1 Perfil de repositórios “não-software”

A análise dos **295 repositórios sem releases** (RQ03) revelou um subgrupo significativo de projetos que não se enquadram no modelo tradicional de desenvolvimento de software:

- Repositórios classificados como `Unknown` (sem linguagem primária detectável) e `Jupyter Notebook` apresentam mediana de releases igual a **0,0**.
- Entre os repositórios mais estrelados sem releases, encontram-se *awesome lists*, guias de entrevista, tutoriais e coleções de referência — projetos de **curadoria de conhecimento** que utilizam o GitHub como plataforma de publicação, não como ferramenta de versionamento de software.
- Esses repositórios apresentam medianas de PRs significativamente menores (**129,0** para Unknown, **88,0** para Jupyter Notebook) em comparação com projetos de engenharia, confirmando um perfil de atividade distinto.

Essa observação levanta uma questão metodológica relevante: estudos empíricos baseados nos repositórios mais estrelados do GitHub devem considerar que aproximadamente **10–15%** da amostra pode não representar projetos de engenharia de software no sentido estrito.

5 Conclusão

5.1 Tomada de decisão

- **Gestão Ativa e Governança:** O alto índice de issues fechadas (mediana de 87,88%) demonstra que projetos de sucesso exigem um esforço contínuo de triagem e manutenção. Equipes que desejam abrir o código de seus sistemas precisam alocar recursos específicos para a gestão da comunidade, garantindo que dúvidas e bugs sejam respondidos rapidamente.

- **Adoção de Stack Tecnológica:** A dominância de Python, TypeScript e JavaScript sugere que a escolha da linguagem primária é um fator estratégico. Projetos que utilizam essas linguagens tendem a atrair um volume maior de contribuições externas, o que é vital para a sustentabilidade do repositório a longo prazo.

- **Maturidade como Estratégia de Longo Prazo:** Com a mediana de idade dos projetos superando os 8 anos, fica evidente que o sucesso no open-source é um processo cumulativo. O planejamento do ciclo de vida do software deve contemplar um amadurecimento gradual, sem a expectativa de tração imediata.

- **Adequação de Processos de Release:** A variação na frequência de releases (especialmente a alta frequência em projetos Go e TypeScript) indica que a adoção de práticas de integração e entrega contínuas (CI/CD) deve ser moldada conforme a cultura do ecossistema da linguagem escolhida.

5.2 Sugestões futuras

- **Filtragem Heurística de Repositórios:** Como os dados revelaram uma porção significativa de

repositórios de “não-software” (como awesome lists e tutoriais), trabalhos futuros podem desenvolver e aplicar filtros automáticos para excluir curadorias de conhecimento, refinando as métricas estritamente para engenharia de software.

- **Análise de Qualidade e Tempo (Lead Time):** Em vez de medir apenas o volume absoluto de Pull Requests e Issues, estudos futuros podem investigar o tempo médio para o fechamento (Merge/Close) desses artefatos, avaliando a agilidade e a eficiência das equipes de manutenção.

- **Impacto da Automação (CI/CD):** Investigar a correlação entre a presença de pipelines automatizados (como GitHub Actions) e a frequência de lançamentos (releases) ou a taxa de aceitação de contribuições externas.

- **Expansão da Amostra:** Ampliar a coleta para os 10.000 repositórios mais populares para verificar se as tendências de maturidade e gestão de issues se mantêm em camadas menos visíveis da plataforma.

5.3 Resultado conclusivo sucinto

Este estudo analisou os 1.000 repositórios mais populares do GitHub (coleta em 13/03/2026) e encontrou evidências de que esses projetos são, em sua maioria: **maduros** (mediana de 8,33 anos), **colaborativos** (mediana de 738 PRs aceitas), **ativamente mantidos** (98,7% atualizados no dia da coleta), com **alta resolução de issues** (mediana de 87,88% fechadas) e **dominados por Python, TypeScript e JavaScript** (47,8% da amostra). A prática de releases formais é adotada pela maioria, mas com polarização significativa (29,5% sem releases vs. 34,2% com 100+). TypeScript se destaca como o ecossistema com maior throughput colaborativo (mediana de 2.528,5 PRs e 157,5 releases). A análise de contribuidores mencionáveis

(mediana de 230) revelou que o tamanho da comunidade é um forte preditor do volume de contribuições, sugerindo retornos crescentes de escala na colaboração open-source.

5.4 Confronto com a literatura

Os resultados empíricos obtidos neste estudo corroboram e atualizam diversas observações presentes na literatura recente (2021–2026) de Engenharia de Software e Mineração de Repositórios de Software (MSR), refletindo o estado da arte do ecossistema do GitHub.

No que tange à maturidade e manutenção (RQ01 e RQ04), Ait, Izquierdo e Cabot (2022) investigaram a taxa de sobrevivência de projetos no GitHub, revelando que grande parte dos repositórios é abandonada em seu primeiro ano e que a probabilidade de sobrevivência a longo prazo é baixa, exigindo períodos ativos e contínuos de desenvolvimento. Nossos dados confirmam empiricamente essa dinâmica: os repositórios que alcançam o topo da popularidade são os "sobreviventes" de longo prazo desse ecossistema competitivo, apresentando uma impressionante mediana de idade de 8,33 anos e uma taxa de atualização quase unânime (98,7% atualizados no dia da coleta).

Em relação ao modelo de colaboração (RQ02 e RQ06), o trabalho de Zhang et al. (2022) fornece um panorama empírico amplo sobre o processamento de *Pull Requests*, destacando esse mecanismo e a gestão de *issues* como o motor central do desenvolvimento moderno em plataformas *pull-based*. A altíssima adoção evidenciada em nossa amostra (mediana de 738 PRs aceitas) e a forte capacidade de fechamento de *issues* (mediana de 87,88%) reforçam a conclusão de Zhang et al. de que ecossistemas populares dependem de uma triagem rigorosa e constante para manter a comunidade colaborativa engajada e reduzir a latência de integração.

Por fim, a constatação da Seção 4.1 — de que 10% a 15% dos repositórios mais populares são focados em curadoria de conhecimento (como *awesome lists*) e não em engenharia de software tradicional — dialoga diretamente com os desafios metodológicos documentados na literatura contemporânea. Como apontado por Wessel et al. (2023) ao filtrarem rigorosamente repositórios para analisar o impacto de automações no processo de *Pull Requests*, assumir que qualquer repositório estrelado possui o mesmo perfil de desenvolvimento e ciclo de *releases* (RQ03) pode enviesar análises de engenharia de software. Nossos achados quantificam esse fenômeno no atual top 1.000, reiterando a necessidade premente de aplicar heurísticas de exclusão na seleção de amostras em estudos empíricos modernos.

Referências

AIT, A.; IZQUIERDO, J. L. C.; CABOT, J. An empirical study on the survival rate of GitHub projects. In: INTERNATIONAL CONFERENCE ON MINING SOFTWARE REPOSITORIES (MSR), 19., 2022, Pittsburgh. **Proceedings** [...]. New York: ACM, 2022. p. 365-375.

WESSEL, M. et al. GitHub Actions: The Impact on the Pull Request Process. **Empirical Software Engineering**, v. 28, n. 6, p. 1-38, 2023.

ZHANG, X. et al. Pull request latency explained: an empirical overview. **Empirical Software Engineering**, v. 27, n. 6, p. 1-35, 2022.